Lavorare con i Dati Usando l'AI

Sessione 2 di 3: Cercare per Significato

Chi sono

Federico Rios

- AI Engineer presso **aidea**
- Docente per **Boolean**
- Ricercatore al **Politecnico di Milano** (fino a poco fa)



Recap: Sessione 1

Cosa abbiamo fatto la volta scorsa

Abbiamo preso del **testo grezzo di ricette** e usato un LLM per estrarne **dati strutturati**.

- Scaricato 13.500 ricette da Epicurious (Hugging Face)
- Esplorato i dati con Python puro
- Sbattuto contro il muro: le regex non bastano per il testo reale
- Usato un LLM per estrarre JSON strutturato, e ha funzionato

L'LLM ci ha dato questo

Dalla stringa `"2¾ tsp. kosher salt, divided, plus more"` :

```
{
  "quantity": 2.75,
  "unit": "tsp",
  "ingredient": "kosher salt",
  "notes": "divided, plus more"
}
```

E i metadati completi della ricetta:

```
{
  "cuisine": "American",
  "difficulty": "medium",
  "techniques": ["roast", "sauté", "reduce"],
  "dietary": [],
  "total_time_minutes": 120
}
```

Il nuovo problema

Abbiamo 13.500 ricette. Come le cerchiamo?

Immagina che un utente chieda:

```
"Voglio qualcosa con pomodoro e basilico"
```

Ricerca per parole chiave: cerchiamo le parole "pomodoro" e "basilico" nel testo.

Funziona... finché non funziona più.

Quando la ricerca per keyword fallisce

Query	Il problema
`"tomato"`	Non trova ricette che dicono **"marinara sauce"**, che *è* pomodoro
`"quick weeknight dinner"`	**Zero risultati**, nessuna ricetta contiene queste parole
`"lemon juice"`	Non trova ricette che scrivono **"juice of 2 lemons"**

Il punto: la ricerca per keyword confronta **parole**, non **significato**.

Ci serve un modo per cercare in base a **cosa intende l'utente**, non a cosa ha scritto letteralmente.

Vettori ed Embeddings

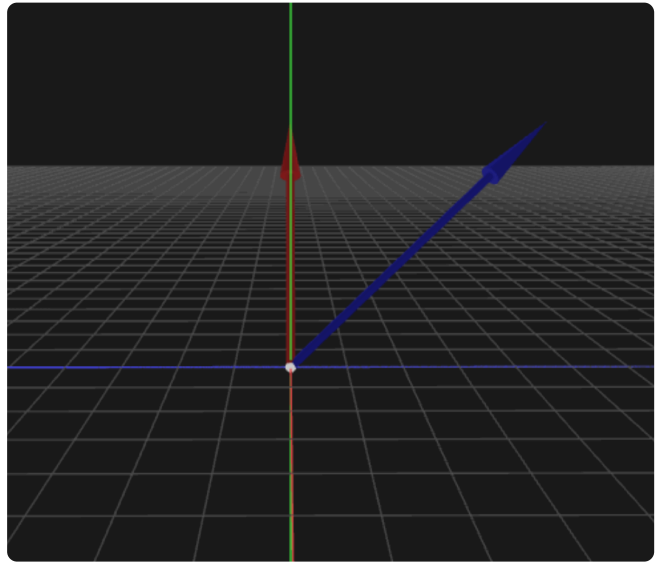
Cos'è un vettore?

Un vettore è semplicemente una **lista di numeri** che rappresenta una posizione nello spazio.

Esempio 2D: il punto (3, 4) è un vettore, ci dice *dove* si trova qualcosa.

Esempio 3D: (3, 4, 5), stessa idea, una dimensione in più.

Fin qui possiamo visualizzarlo. Ma cosa succede quando le dimensioni diventano **centinaia**?

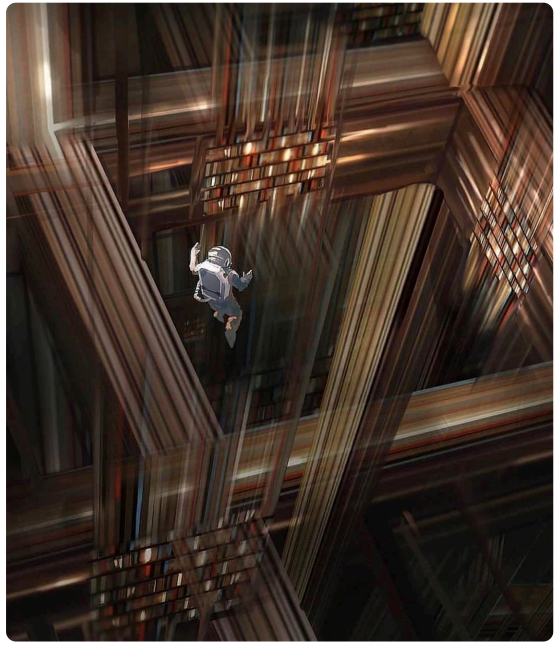


Embedding

Embedding: (0.021, -0.134, 0.892, ...), un vettore con **centinaia o migliaia** di dimensioni.

Non possiamo più visualizzarlo, ma la matematica funziona allo stesso modo.

Se possiamo trasformare il testo in vettori, possiamo **misurare quanto due testi sono vicini**, misurando la distanza tra i loro vettori.



Perché rappresentare il testo come numeri?

I computer non capiscono "chicken" o "pollo", ma sanno calcolare **distanze**.

Se abbiamo un buon modo per convertire testo → vettori:

- **"chicken"** e **"pollo"** → vettori **vicini** (stesso significato)
- **"chicken"** e **"algebra"** → vettori **lontani** (nessuna relazione)
- **"marinara sauce"** e **"tomato"** → vettori **vicini** (concetti correlati)

Questa mappatura da testo a vettori si chiama **embedding**.

Un modello di embedding viene addestrato su enormi quantità di testo per imparare queste relazioni.

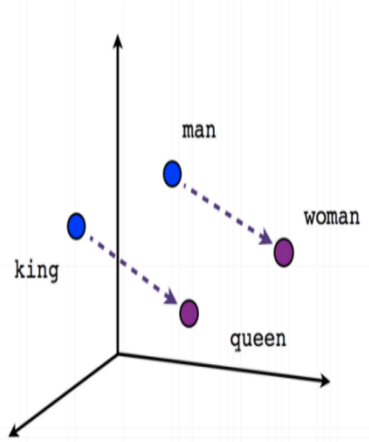
La scoperta sorprendente

Quando i ricercatori hanno addestrato i primi modelli di word embedding (Word2Vec, 2013), hanno scoperto qualcosa che **nessuno si aspettava**.

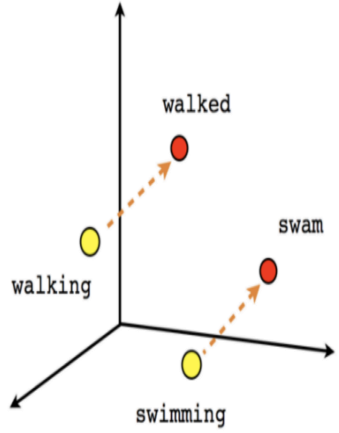
I vettori avevano imparato **relazioni aritmetiche**:

- `king - man + woman ≈ queen`
- `Paris - France + Italy ≈ Rome`
- `walking - walk + swim ≈ swimming`

Nessuno l'aveva programmato. Queste relazioni sono **emerse da sole** dal training.



Male-Female



Verb tense

Cosa significa per noi

I modelli di embedding moderni (come quelli di OpenAI) vanno molto oltre: possono rappresentare **intere frasi**, non solo singole parole.

- `"a light summer recipe with fresh herbs"` → un vettore
- `"quick weeknight dinner"` → un vettore
- `"Crispy Salt and Pepper Potatoes, 7 ingredients, 30 minutes"` → un vettore vicino

Ora possiamo **cercare per significato**: trasformiamo la query dell'utente in un vettore e troviamo i vettori delle ricette più vicini.

Questo si chiama **ricerca semantica**.

Vediamolo in pratica

(notebook 1: embeddings e cosine similarity)

Perché ci serve un vector database?

NumPy funziona, ma...

Quello che abbiamo appena fatto, calcolare la cosine similarity con NumPy, funziona benissimo su 1.000 ricette.

Ma con **100.000 ricette**? O **1 milione**?

- Ogni query confronta **tutti** i vettori → O(n)
- Tutti i vettori devono stare **in memoria**
- Nessuna persistenza: gli embedding si perdono quando il notebook si riavvia
- Nessun filtro (es. "solo ricette vegetariane")

Un **vector database** risolve tutto questo:

- Algoritmi di **Approximate Nearest Neighbor** (ANN) → veloce anche su larga scala
- **Storage persistente**: i dati sopravvivono ai riavvii
- **Filtri sui metadati**: combina ricerca semantica e filtri

Qdrant

Useremo **Qdrant**, un vector database open-source.

- Si avvia in locale con un singolo `docker compose up`
- Ha una **dashboard web** su `localhost:6333/dashboard`
- Il client Python è semplice e intuitivo
- Possiamo vedere i nostri vettori, interrogarli ed esplorarli, tutto dal browser

Carichiamo i nostri embedding delle ricette in Qdrant e interroghiamolo.

Proviamolo

(notebook 2: Qdrant)

Dalla ricerca all'assistente

E se un chatbot potesse cercare per noi?

Ora abbiamo una funzione che prende una query e restituisce le ricette più rilevanti.

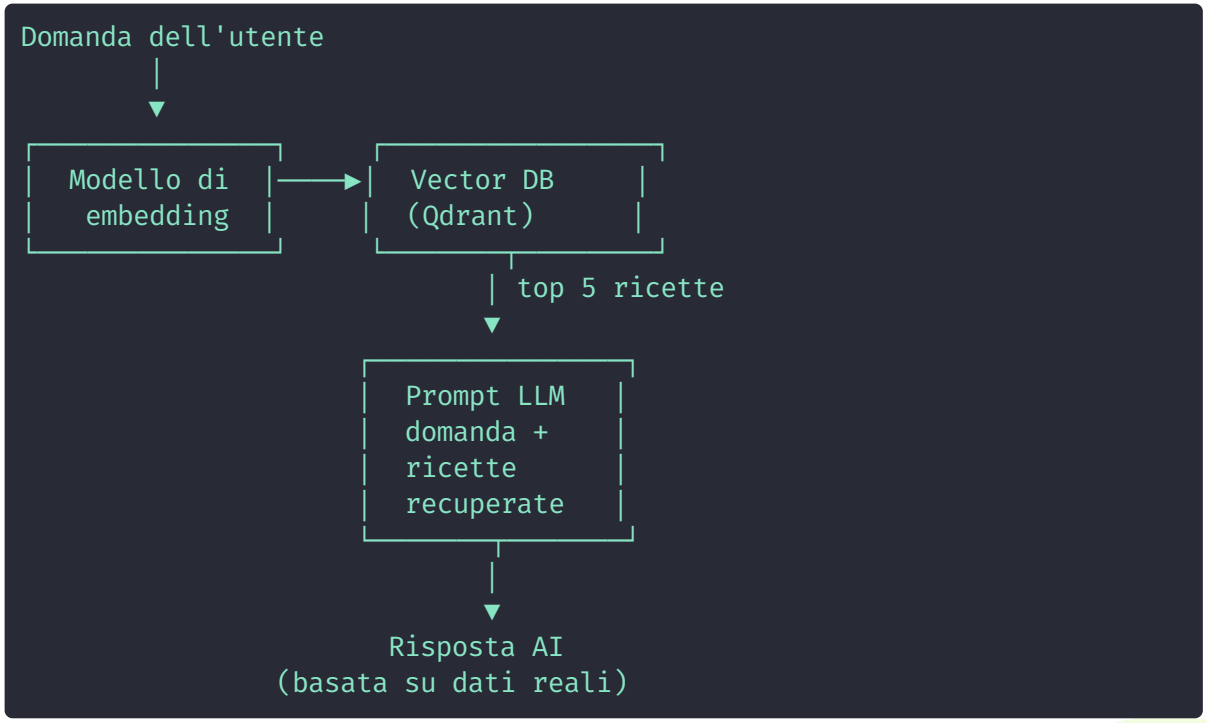
Possiamo costruire un **chatbot** che usa questo meccanismo ad ogni domanda:

1. L'utente chiede: *"Cosa posso cucinare con il pollo avanzato?"*
2. La domanda viene **trasformata in embedding** e usata per cercare ricette simili
3. Le 5 ricette più rilevanti vengono recuperate da Qdrant
4. Ricette + domanda vengono passate come **contesto** all'LLM, che genera una risposta utile

Questo pattern si chiama **RAG**, Retrieval-Augmented Generation.

L'LLM non inventa: prima **recupera** dati reali, poi **genera** una risposta basata su quei dati.

L'architettura



Nella **Sessione 3** andremo oltre: l'LLM **deciderà da solo** quando cercare, usando il function calling.

Costruiamolo

(notebook 3: RAG)